DocuVerify: Identity Document Verification System

DocuVerify is a full-stack web application designed to automatically extract, cross-verify, and score identity documents (like Passports, Driving Licenses, PAN Cards, and Aadhaar Cards) to determine the eligibility of an applicant.

It leverages advanced AI models (Google Gemini API) to extract structured text from images and PDFs, falling back to traditional OCR (Tesseract) if needed. The extracted data is then cross-referenced using fuzzy string matching to generate an overall consistency score.

🚀 Features

- **Multi-Document Upload:** Upload multiple identity documents simultaneously (JPEG, PNG, PDF).
- **AI-Powered Data Extraction:** Uses the latest `gemini-2.5-flash` model via the Google GenAI SDK to accurately pull Name, Date of Birth, Address, and City from complex identity documents.
- **OCR Fallback System:** Fallback mechanism built using PyTesseract, `pdfplumber`, and spaCy for environments where Gemini is unavailable or quotas are exceeded.
- **Intelligent Cross-Verification:** Compares extracted fields across different documents using `thefuzz` library, calculating Levenshtein distance for fuzzy matching (handling slight typos or format variations).
- **Beautiful UI/UX:** A modern, responsive React frontend built with Vite, Tailwind CSS v4, and Lucide React icons.

🛠 Tech Stack

Backend

- **Framework:** FastAPI (Python 3.11+)
- **Database:** SQLite with SQLAlchemy ORM
- **AI/Extraction:** `google-genai` SDK, PyTesseract, Pillow, pdfplumber, spaCy
- **Comparison Logic:** `thefuzz`

Frontend

- **Framework:** React 18, Vite
- **Styling:** Tailwind CSS v4
- **Routing:** React Router DOM
- **Icons:** Lucide React
- **HTTP Client:** Axios

⚙ Setup & Installation

Prerequisites

- **Python 3.11+**
- **Node.js 18+**
- **Tesseract OCR** (Optional but recommended for fallback extraction)

  - *Windows:* Download the installer from UB-Mannheim/tesseract and ensure it's added to your system PATH.
- **Google Gemini API Key:** Get one from Google AI Studio.

1. Backend Setup

1. Open your terminal and navigate to the backend folder:

```
cd DocuVerify/backend
```

2. Create and activate a Python virtual environment:

```
python -m venv venv

# Windows:
.\venv\Scripts\activate
# macOS/Linux:
source venv/bin/activate
```

3. Install dependencies:

```
pip install -r requirements.txt
```

4. Configure Environment Variables: Create a `.env` file inside the `backend` folder and add your Gemini API key:

```
GEMINI_API_KEY=your_google_gemini_api_key_here
EXTRACTION_BACKEND=gemini
```

5. Start the FastAPI server:

```
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

The backend will now be running at `http://127.0.0.1:8000`.

2. Frontend Setup

1. Open a new terminal and navigate to the frontend folder:

```
cd DocuVerify/frontend
```

2. Install Node dependencies:

```
npm install
```

3. Start the Vite development server:

```
npm run dev
```

The frontend will now be running (usually at `http://localhost:5174`).

🧠 How It Works

1. **Upload Session Initiation (`/upload`)**: When you drop files in the UI, the frontend sends them to the FastAPI backend. The backend creates a unique `VerificationSession` and stores the raw images/PDFs in the `uploads/` directory, saving metadata to the SQLite database.

2. **Extraction Phase (`/verify`)**:

   - The backend reads each document and sends the image data to Google's **Gemini 2.5 Flash** model with a strict prompt forcing it to return structured JSON `{"name": "...", "dob": "...", "address": "...", "city": "..."}`.
   - If Gemini fails (e.g., rate limit exceeded), it falls back to **Tesseract OCR**, scanning the document line-by-line using Regex and `spaCy` NLP to guess the fields.

3. **Cross-Verification Phase (`/verify`)**:

   - Once data is extracted from all documents, the `ComparisonService` kicks in.
   - It pairs every document against every other document (e.g., Doc A vs Doc B, Doc B vs Doc C).
   - Using fuzzy string matching, it calculates a similarity percentage for Name, DOB, Address, and City.
   - The overall average of these matches becomes the **Overall Consistency Score**.

4. **Results Phase (`/results`)**:

   - If the Consistency Score is **> 80%**, the applicant is marked as **Eligible**.
   - If the score is between **70% - 80%**, it is marked for **Manual Review**.
   - If the score is **< 70%**, the applicant is **Rejected**.
   - The frontend fetches this data and renders beautiful circular gauges, field-by-field breakdowns, and deduplicated comparison tables.

📂 Project Structure

```
DocuVerify/
├── backend/
```

```
│   ├── app/
│   │   ├── main.py                # FastAPI application entry point
│   │   ├── config.py              # Environment variables config
│   │   ├── database.py            # SQLite DB connection
│   │   ├── models/                # SQLAlchemy ORM Models
│   │   ├── schemas/               # Pydantic validation schemas
│   │   ├── routers/               # API Endpoints (upload, verify, results)
│   │   └── services/              # Core business logic
│   │       ├── gemini_service.py   # AI Extraction
│   │       ├── ocr_service.py      # Tesseract Extraction Fallback
│   │       ├── comparison_service.py # Fuzzy matching
│   │       └── eligibility_service.py# Threshold logic
│   ├── uploads/                   # Local storage for uploaded docs
│   ├── .env                       # Secret keys
│   └── requirements.txt           # Python dependencies
│
└── frontend/
    ├── src/
    │   ├── App.jsx                # React Router setup
    │   ├── main.jsx               # React DOM entry
    │   ├── index.css              # Tailwind directives
    │   └── views/                 # UI Components
    │       ├── UploadView.jsx      # Drag & Drop Interface
    │       ├── ProcessingView.jsx  # Loading & Trigger Verification
    │       └── ResultsView.jsx     # Dashboard showing scores & data
    ├── vite.config.js             # Vite config (Handles /api proxying to
backend)
    └── package.json               # Node dependencies
```

License

This project is licensed under the MIT License.